Understanding Generative Adversarial Networks from a mathematical view

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

December 8, 2025

Feynman's words

What I cannot create, I do not understand.

Outline

The Generative Modeling Problem

Given a dataset $\{x_i\}_{i=1}^n$ sampled from an unknown distribution $p_{data}(x)$, how to learn a model that can generate new samples from $p_{data}(x)$?

- Explicit modeling: Learn $p_\theta(x)$ directly
  - Maximum Likelihood Estimation (MLE)
  - Normalizing Flows
- Implicit modeling: Learn a generator $G_\theta(z)$ that maps noise to data
  - Generative Adversarial Networks (GANs)
  - Diffusion Models
  - Variational Autoencoders (VAEs)

The Adversarial Training Paradigm

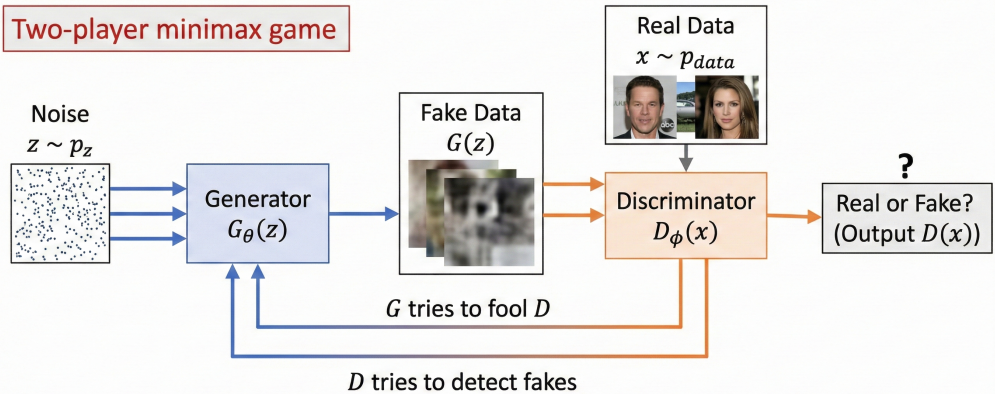
The key insight of GANs: Two-player minimax game

- **Generator** $G_\theta(z)$: Maps noise $z \sim p_z$ to data space
- **Discriminator** $D_\phi(x)$: Distinguishes real from fake data
- **Objective**: Generator tries to fool discriminator, discriminator tries to detect fakes

The minimax objective:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))] \tag{1}$$

The Adversarial Training Paradigm



$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$$

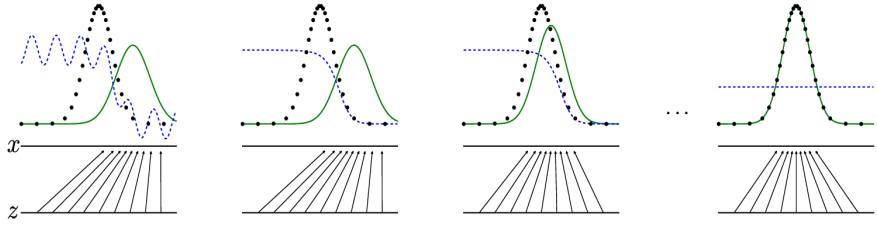


Figure 1: Generative adversarial nets are trained by simultaneously updating the **d**iscriminative distribution (D, blue, dashed line) so that it discriminates between samples from the data generating distribution (black, dotted line) $p_{\boldsymbol{x}}$ from those of the **g**enerative distribution p_g (G) (green, solid line). The lower horizontal line is the domain from which $\boldsymbol{z}$ is sampled, in this case uniformly. The horizontal line above is part of the domain of $\boldsymbol{x}$. The upward arrows show how the mapping $\boldsymbol{x} = G(\boldsymbol{z})$ imposes the non-uniform distribution p_g on transformed samples. G contracts in regions of high density and expands in regions of low density of p_g. (a) Consider an adversarial pair near convergence: p_g is similar to p_{data} and D is a partially accurate classifier. (b) In the inner loop of the algorithm D is trained to discriminate samples from data, converging to $D^*(\boldsymbol{x}) = \frac{p_{\text{data}}(\boldsymbol{x})}{p_{\text{data}}(\boldsymbol{x}) + p_g(\boldsymbol{x})}$. (c) After an update to G, gradient of D has guided $G(\boldsymbol{z})$ to flow to regions that are more likely to be classified as data. (d) After several steps of training, if G and D have enough capacity, they will reach a point at which both cannot improve because $p_g = p_{\text{data}}$. The discriminator is unable to differentiate between the two distributions, i.e. $D(\boldsymbol{x}) = \frac{1}{2}$.

北京雁栖湖
应用数学研究院
Beijing Institute
of Mathematical
Sciences and Applications

Why Adversarial Training?

Traditional generative models face challenges:

- **Explicit likelihood**: Hard to compute for complex distributions
- **Mode collapse**: Tend to generate similar samples
- **Blurry samples**: MLE often produces average-looking samples

GANs offer advantages:

- No explicit likelihood: Avoid normalization issues
- Sharp samples: Adversarial training encourages realistic details
- Flexible architecture: Can use any differentiable network

Outline

The Minimax Game Formulation

For fixed generator G, the optimal discriminator is:

$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \tag{2}$$

Proof.

The discriminator loss for fixed G:

$$V(D, G) = \int_x p_{data}(x) \log D(x) + p_g(x) \log(1 - D(x)) dx \tag{3}$$

Taking derivative w.r.t. $D(x)$ and setting to zero:

$$\frac{p_{data}(x)}{D(x)} - \frac{p_g(x)}{1 - D(x)} = 0 \Rightarrow D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \tag{4}$$

□

Global Optimum Analysis

When $p_g = p_{data}$, the optimal discriminator becomes:

$$D_G^*(x) = \frac{1}{2} \tag{5}$$

The global minimum of the generator loss is:

$$C(G) = \max_D V(D, G) = -\log(4) + 2 \cdot JSD(p_{data} \| p_g) \tag{6}$$

where JSD is the Jensen-Shannon divergence.

Proof.

Substituting D_G^* into $V(D, G)$:

$$\begin{aligned} C(G) &= \mathbb{E}_{x \sim p_{data}}[\log \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}] + \mathbb{E}_{x \sim p_g}[\log \frac{p_g(x)}{p_{data}(x) + p_g(x)}] \\ &= -\log(4) + KL(p_{data} \| \frac{p_{data} + p_g}{2}) + KL(p_g \| \frac{p_{data} + p_g}{2}) \\ &= -\log(4) + 2 \cdot JSD(p_{data} \| p_g) \end{aligned} \tag{7}$$

Convergence Analysis

Theorem: If G and D have enough capacity, and at each step the discriminator is allowed to reach its optimum given G, then p_g converges to p_{data}.

- Sufficient capacity: Networks can represent the optimal functions
- Perfect discriminator: D reaches optimum at each step
- Generator updates: G is updated to minimize $C(G)$

However, in practice:

- Networks have limited capacity
- Discriminator is not trained to optimality
- Training dynamics can be unstable

BIMSA 北京雁栖湖应用数学研究院 Beijing Institute of Mathematical Sciences and Applications

Outline

Original GAN Loss

The original GAN uses the binary cross-entropy loss:

$$\mathcal{L}_{GAN} = \mathbb{E}_{x \sim p_{data}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))] \tag{8}$$

Problems with original loss:

- Gradient vanishing: When D is too good, gradients vanish
- Mode collapse: Generator finds one mode and sticks to it
- Training instability: Oscillations between G and D

Non-saturating loss (alternative for generator):

$$\mathcal{L}_G^{NS} = -\mathbb{E}_{z \sim p_z}[\log D(G(z))] \tag{9}$$

Outline

WGAN: What goes wrong with vanilla GANs?

- Objective uses **Jensen–Shannon (JS)** divergence between p_θ and $p_{\mathcal{D}}$.
- When supports are disjoint or lie on low-dimensional manifolds, JS is **maxed out** and gradients vanish.
- Symptoms: mode collapse, training instability, sensitivity to architecture and hyperparameters.

WGAN: Wasserstein-1 (Earth-Mover) distance

$$\mathcal{W}(p_\theta, p_{\mathcal{D}}) \;=\; \inf_{\gamma \in \Pi(p_\theta, p_{\mathcal{D}})} \mathbb{E}_{(\mathbf{x},\tilde{\mathbf{x}})\sim\gamma}\big[\,\|\mathbf{x}-\tilde{\mathbf{x}}\|\,\big], \tag{10}$$

where Π are couplings with marginals p_θ and $p_{\mathcal{D}}$. **Why it helps:**

- Provides *meaningful* distances even when supports do not overlap.
- Changes *continuously* with generator parameters θ.
- Leads to **usable gradients** almost everywhere.

WGAN: Kantorovich–Rubinstein duality

$$\mathcal{W}(p_\theta, p_{\mathcal{D}}) \;=\; \sup_{\|f\|_{\mathrm{Lip}} \le 1} \Big(\mathbb{E}_{\mathbf{x} \sim p_{\mathcal{D}}}[f(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim p_\theta}[f(\mathbf{x})] \Big). \tag{11}$$

WGAN idea

Learn a **1-Lipschitz critic** $\mathcal{D}$ to approximate the supremum and minimize $\mathcal{W}$ by moving p_θ.

WGAN: Optimization problem

Critic (*not a classifier*):

$$\max_{\psi\,:\,\|\mathcal{D}\|_{\mathrm{Lip}}\leq 1} \mathbb{E}_{\mathbf{x}\sim p_{\mathcal{D}}}[\mathcal{D}(\mathbf{x})] - \mathbb{E}_{\mathbf{z}\sim p(\zeta)}[\mathcal{D}(\mathcal{G}(\mathbf{z}))]. \tag{12}$$

Generator:

$$\min_{\theta}\ \mathbb{E}_{\mathbf{z}\sim p(\zeta)}[\mathcal{D}(\mathcal{G}(\mathbf{z}))]. \tag{13}$$

- The critic's output is a **score** (real number), not a probability.
- Enforcing 1-Lipschitzness is *crucial* (see next slide).

WGAN: How to enforce 1-Lipschitz?

Weight clipping (original WGAN)

- Constrain each weight to $[-c, c]$.
- Simple but can underfit or cause gradient issues.

Gradient penalty (WGAN-GP)

$$\lambda \, \mathbb{E}_{\hat{\mathbf{x}}=t\mathbf{x}+(1-t)\tilde{\mathbf{x}}} \big(\|\nabla_{\hat{\mathbf{x}}} \mathcal{D}(\hat{\mathbf{x}})\|_2 - 1 \big)^2. \qquad (14)$$

- Encourages $\|\nabla \mathcal{D}\| \approx 1$ on lines between real and fake.
- Typically far more stable than clipping.

Least Squares GAN (LSGAN)

LSGAN uses the least squares loss:

$$\min_D \mathcal{L}_D^{LS} = \frac{1}{2}\mathbb{E}_{x\sim p_{data}}[(D(x)-1)^2] + \frac{1}{2}\mathbb{E}_{z\sim p_z}[(D(G(z)))^2] \tag{15}$$

$$\min_G \mathcal{L}_G^{LS} = \frac{1}{2}\mathbb{E}_{z\sim p_z}[(D(G(z))-1)^2] \tag{16}$$

Advantages:

- Stable gradients: No vanishing gradient problem
- Better quality: Produces higher quality samples
- Faster convergence: More stable training dynamics

Outline

DCGAN: Deep Convolutional GAN

Key architectural guidelines:

- **Replace pooling** with strided convolutions
- **Remove fully connected layers** (except first/last)
- **Batch normalization** in both G and D
- **ReLU** in generator, **LeakyReLU** in discriminator
- **Remove bias** from batch norm layers

Generator architecture:

- Input: 100-dim noise z
- Project to $4 \times 4 \times 1024$ feature maps
- Upsample through transposed convolutions
- Output: $64 \times 64 \times 3$ RGB image

Progressive Growing of GANs

Train GANs progressively from low to high resolution:

- Start with 4×4 images
- Gradually add layers for higher resolution
- Smooth transition between resolutions

Benefits:

- Stable training: Easier to train at low resolution
- High quality: Can generate very high resolution images
- Faster convergence: Progressive training is more efficient

Key techniques:

- Fade-in new layers gradually
- Use minibatch standard deviation
- Equalized learning rate

StyleGAN: Style-Based Generator

StyleGAN introduces style-based architecture:

- **Mapping network**: Transforms z to intermediate latent w
- **Style modulation**: Each layer can be modulated by style
- **Noise injection**: Adds stochastic variation
- **Progressive growing**: Builds up resolution gradually

Key innovations:

$$y = (w \cdot s) \cdot x + b \tag{17}$$

where w is the style vector, s is the style scale, x is the feature map.

Benefits:

- Better control: Separate content and style
- Higher quality: More realistic generation
- Interpolation: Smooth latent space

Outline

Common Training Problems

Mode Collapse:

- Generator produces limited variety of samples
- Discriminator becomes too strong
- Generator finds one "safe" mode

Training Instability:

- Losses oscillate wildly
- Generator and discriminator don't converge
- Sensitive to hyperparameters

Vanishing Gradients:

- Discriminator becomes too good
- Generator receives no useful gradients
- Training stagnates

Solutions to Training Problems

For Mode Collapse:

- Unrolled GANs: Look ahead in discriminator updates
- Minibatch discrimination: Encourage diversity
- Feature matching: Match feature statistics

For Training Instability:

- Two time-scale update rule: Different learning rates
- Spectral normalization: Constrain discriminator
- Gradient penalty: Enforce Lipschitz constraint

For Vanishing Gradients:

- Wasserstein loss: More stable gradients
- Least squares loss: Better gradient properties
- Hinge loss: Alternative to cross-entropy

Outline

Quantitative Evaluation Metrics

Inception Score (IS):

$$IS = \exp(\mathbb{E}_{x \sim p_g}[KL(p(y|x) \| p(y))]) \tag{18}$$

where $p(y|x)$ is the class probability from Inception network.

Fréchet Inception Distance (FID):

$$FID = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}) \tag{19}$$

where (μ_r, Σ_r) and (μ_g, Σ_g) are mean and covariance of real and generated features.

Precision and Recall:

- **Precision**: Quality of generated samples
- **Recall**: Coverage of real data distribution

Qualitative Evaluation Methods

Interpolation:

- Linear interpolation in latent space
- Should produce smooth transitions
- Tests continuity of learned manifold

Arithmetic in Latent Space:

- $G(z_1) + G(z_2) - G(z_3) \approx G(z_1 + z_2 - z_3)$
- Tests linear structure of latent space

Truncation Trick:

- Vary the magnitude of latent vectors
- Trade-off between quality and diversity
- Useful for controlling generation

Outline

北京雁栖湖应用数学研究院
Beijing Institute of Mathematical Sciences and Applications

R3GAN in one picture: fix the landscape & the dynamics

Goal. Make GAN a *convergent* game with strong fidelity at 1-NFE.
Two steps:

1. **Relativistic pairing (RpGAN)** improves the *loss landscape* by comparing real vs. fake relatively:

$$L_{\mathrm{Rp}}(\theta,\psi) = \mathbb{E}_{x\sim p_{\mathcal{D}},\, z\sim p(\zeta)}\, f\big(D_\psi(G_\theta(z)) - D_\psi(x)\big),$$

   which anchors decision boundaries around each real sample and mitigates mode dropping.

2. **Zero-centered gradient penalties (R1+R2)** fix the *training dynamics*:

$$\mathrm{R1}(\psi) = \tfrac{\gamma}{2}\,\mathbb{E}_{x\sim p_{\mathcal{D}}}\big[\|\nabla_x D_\psi(x)\|_2^2\big], \quad \mathrm{R2}(\theta,\psi) = \tfrac{\gamma}{2}\,\mathbb{E}_{x\sim p_\theta}\big[\|\nabla_x D_\psi(x)\|_2^2\big].$$

Why RpGAN helps: relative scoring removes bad minima

Classic f-GAN:

$$L(\theta, \psi) = \mathbb{E}_z[f(D_\psi(G_\theta(z)))] + \mathbb{E}_x[f(-D_\psi(x))].$$

Mode dropping corresponds to spurious local minima/flat regions.

RpGAN:

$$L_{\mathrm{Rp}}(\theta, \psi) = \mathbb{E}_{x,z}\, f\big(D_\psi(G_\theta(z)) - D_\psi(x)\big).$$

- Each real x contributes a local margin against nearby fakes $\Rightarrow$ fewer mode-drop basins.
- *However*, unregularized RpGAN can still diverge under gradient descent (unstable dynamics).

R1+R2: enforcing the correct fixed point (zero slope at equilibrium)

At equilibrium $p_\theta = p_{\mathcal{D}}$ we want the critic's spatial gradient to *vanish*:

$$\nabla_x D_\psi(x) = 0 \quad \text{for } x \in \text{supp}(p_\theta) = \text{supp}(p_{\mathcal{D}}).$$

Zero-centered penalties achieve exactly this:

$$\text{R1}(\psi) = \frac{\gamma}{2}\,\mathbb{E}_{x\sim p_{\mathcal{D}}}\big[\|\nabla_x D_\psi(x)\|^2\big], \quad \text{R2}(\theta,\psi) = \frac{\gamma}{2}\,\mathbb{E}_{x\sim p_\theta}\big[\|\nabla_x D_\psi(x)\|^2\big].$$

Contrast with WGAN-GP ("1-GP"): encourages $\|\nabla_x D\| \approx 1$ along real–fake lines, which *does not* enforce the zero-slope fixed point and can sustain rotational dynamics near the optimum.

Local convergence via Jacobian of the game dynamics (informal)

Define the training field

$$v(\theta,\psi) = \begin{pmatrix} -\nabla_\theta L_{\mathrm{Rp}}(\theta,\psi) \\ \nabla_\psi L_{\mathrm{Rp}}(\theta,\psi) + \nabla_\psi(\mathrm{R1}+\mathrm{R2}) \end{pmatrix}, \qquad J = \nabla v(\theta^*,\psi^*).$$

First-order update $(\theta,\psi) \leftarrow (\theta,\psi) + h\, v(\theta,\psi)$.

Claim (informal): With RpGAN + **R1+R2**, J has eigenvalues with negative real parts near (θ^*,ψ^*) for small h:

$$\Re\,\sigma(J) < 0 \quad \Longrightarrow \quad \text{linearized stability (no cycling).}$$

Intuition: 0-GP damps the critic blocks (adds positive-definite terms), kills the rotational component that causes oscillation, and aligns the fixed point with $\nabla_x D = 0$.

Putting it together: the R3GAN objective & loop

Critic loss (one minibatch):

$$\mathcal{L}_{\text{critic}} = -\frac{1}{m}\sum_{i=1}^{m} f\big(D_\psi(\tilde{x}_i) - D_\psi(x_i)\big) + \frac{\gamma}{2}\Big(\|\nabla D_\psi(x_i)\|^2 + \|\nabla D_\psi(\tilde{x}_i)\|^2\Big),$$

where $\tilde{x}_i = G_\theta(z_i)$.

Generator loss:

$$\mathcal{L}_{\text{gen}} = -\frac{1}{m}\sum_{i=1}^{m} f\big(D_\psi(G_\theta(z_i)) - D_\psi(x_i)\big).$$

Recipe: several critic steps per gen step; Adam with β_1=0; moderate γ; balanced batch sizes.

Why the engineering works (enabled by the math)

Once the *game* is stable, R3GAN drops many ad-hoc tricks and uses a clean backbone:

- **No normalization**; fix-up initialization to keep variance controlled.
- **Bilinear** up/down-sampling; avoid transposed-conv artifacts.
- Per-resolution **Transition** (resample+1x1) + two **1–3–1 residual** blocks.
- **Grouped conv** + **inverted bottlenecks** for efficient width.
- Symmetric G/D; LeakyReLU; small LR for critic.

Outcome: strong 1-NFE quality (FFHQ/CIFAR/ImageNet) and full mode coverage on StackedMNIST *without* StyleGAN-style tricks.

Takeaways

1. **Landscape:** RpGAN's relative scoring reduces mode-drop minima:

$$L_{\mathrm{Rp}}(\theta, \psi) = \mathbb{E}_{x,z} f(D(G(z)) - D(x)).$$

2. **Dynamics:** Zero-centered penalties make the fixed point correct ($\nabla_x D = 0$) and the Jacobian stable:

$$\mathrm{R1} + \mathrm{R2} \;\Rightarrow\; \Re\, \sigma(J) < 0.$$

3. **Engineering:** with a convergent loss, a minimalist modern backbone achieves SOTA-level 1-NFE results.

Outline

Open Questions

If you are interested in these questions, please feel free to write an email to me (hpp@bimsa.cn) with your comments.

1. **Theoretical understanding**: Why do GANs work despite the minimax game being non-convex?
2. **Mode collapse**: Can we guarantee that GANs will cover all modes of the data distribution?
3. **Evaluation metrics**: How can we better evaluate the quality and diversity of generated samples?
4. **Training stability**: What are the fundamental principles for stable GAN training?
5. **Generalization**: How do GANs generalize beyond the training distribution?

Welcome inputs

Any comments?

Welcome your inputs!